Table of Contents

Introduction **2**

Building Home Module **3**

Building Item Module **7**

Reasons to Use React-Bootstrap 8

Building Shared Modules, Navigation, and Routing **10**

Demo: Building React Components

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT and Gemini.

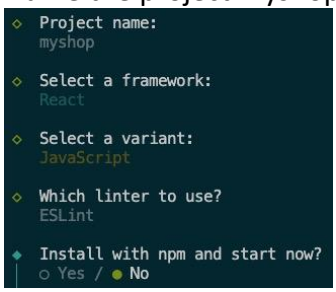
Introduction

This demo guides you to build components in React, including creating and organising a project, handling routing, and using Bootstrap for styling.

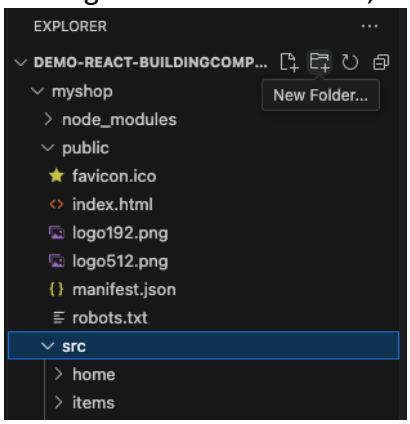
- **Setup Project**: Create a new React project using npm create vite@latest.
- **Organise Project**: Create folders under src for organising your components and pages in modules.
- **Create modules/pages**:
    - **home/HomePage.jsx**: Define a functional component that renders a simple home page.
    - **items/ItemListPage.jsx**: Define a component that displays a list of items in a table.
    - **shared/NavMenu.jsx**: Define a navigation bar using react-bootstrap components like Navbar, Nav, and NavDropdown.
- **Installing packages**:
    - Install react-bootstrap and bootstrap to style your components.
    - Use react-router-dom to manage routes in your app.

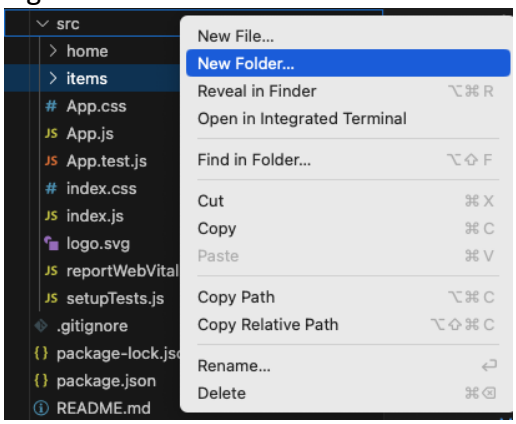
Building Home Module

Go to a folder where you would like to put your myshop project. Run the command:
`npm create vite@latest`
Name the project myshop and select the below options:



Then we create two folders named as “home” and “items” under src. You can do it via clicking the New Folder icon, or right click the src folder and the click New Folder...





There are different ways of organising a React project. Here we use the approach of modules. We use module as a group of pages that serve related functionalities. We create two modules, home and items, each will consist of some pages. A page contains a set of components, and the client-side business logic related to the components.

Create a file “HomePage.jsx” under the home folder.
Here we use the extension jsx, which stands for JavaScript XML.
JSX is a syntax extension for JavaScript commonly used with React. It allows you to write HTML elements and components in a JavaScript file. Differences put simply:

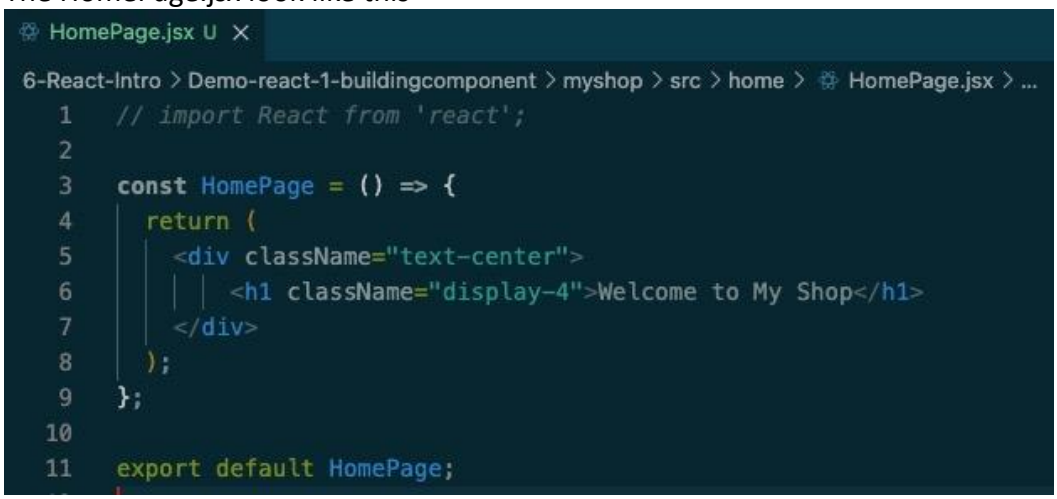
In JS:

```
const element = React.createElement('h1', null, 'Hello, world!');
```

In JSX:

```
const element = <h1>Hello, world!</h1>;$
```

The HomePage.jsx look like this



Line1: This line imports the React library. In React 17 and later, importing React is not strictly necessary for JSX to work, so we comment it out.
Line3: HomePage is a functional component. Functional components are a simpler way to write components in React. They are JavaScript functions that return JSX elements. This component doesn't have any state or lifecycle methods, making it a pure functional component.
The line const HomePage = () => { ... } uses an arrow function to define a React functional component named HomePage.
Arrow functions are a shorthand syntax for defining functions in JavaScript.
Inside the brackets (), there should be the arguments to the function. Now it is empty, which means no arguments are needed.
The curly brackets after the arrow contain the function body, which encloses the logic that the function should execute or what the component should render.
Line4 - Line7: return contains JSX code, which describes what the UI should look like.
: This is a div element with a Bootstrap class text-center. The text-center class centers the text horizontally within this div.
<h1 className="display-4">: This is an h1 element with a Bootstrap class display-4. The display-4 class is a Bootstrap utility class that applies a specific font size and style to the h1 element, making it larger and bolder, typically used for prominent headings.
Line11: This line exports the HomePage component as the default export from the module. This means that when another file imports from this module, it will get the HomePage component by default.

After this, we need to use the HomePage in the App.jsx. Change it as:

App.jsx

src > App.jsx > ...

```
1  import HomePage from './home/HomePage'
2  import './App.css'
3
4  function App() {
5    return (
6      <div className="App">
7       <HomePage />
8      </div>
9    )
10 }
11
12 export default App
```

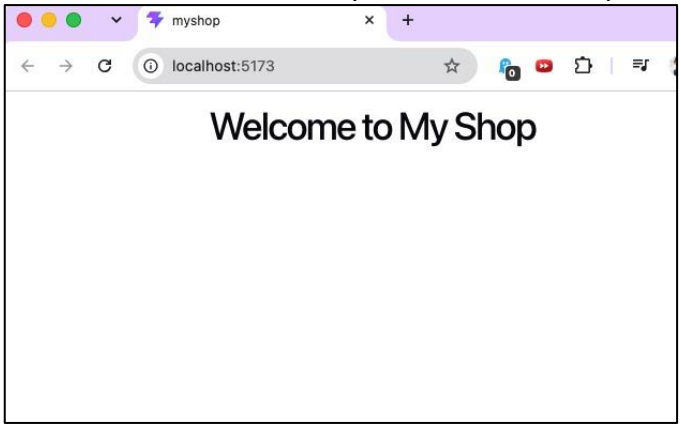
Line1 imports the HomePage.
Line2 imports the App.css used for styling the App component.
Line 4 defines a functional component named App, which returns a jsx component with the class name App, applying styles from App.css. The App class div uses the imported <HomePage /> component.

Function Declarations (function) vs. Arrow Functions (const)
A minor operational difference between function and const arrow functions is **hoisting**:

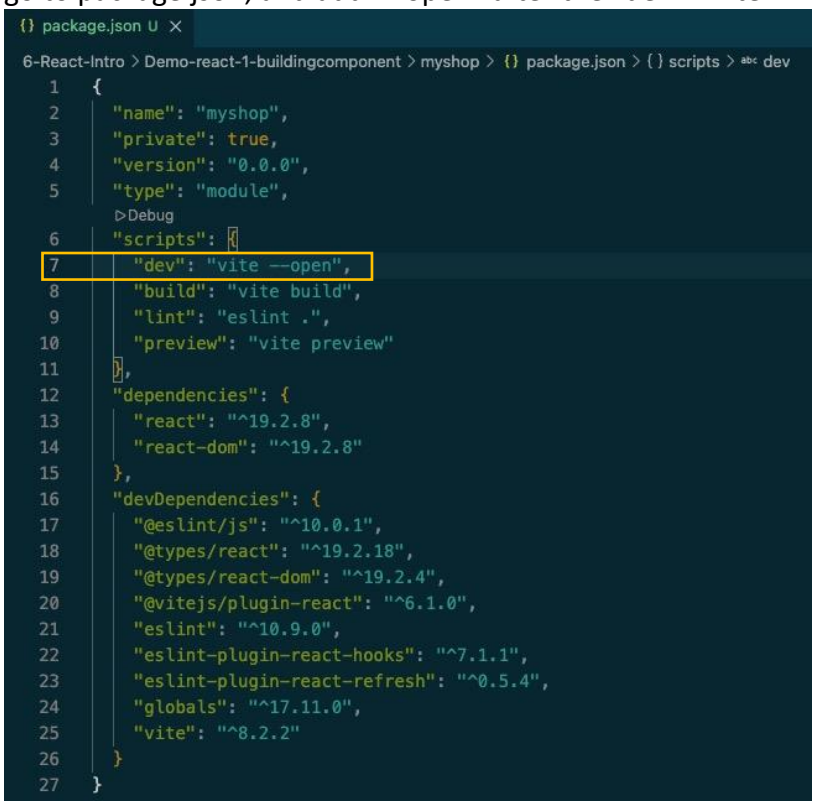
- **function declarations** are hoisted to the top of their scope, meaning they can theoretically be referenced before their definition in the code.
- **const arrow functions** are not initialised until execution reaches their definition, requiring them to be declared before they are called.

Now go to the myshop folder (cd myshop) from the terminal and run with
npm install
npm run dev

Then click the shown URL, you will see the simple home page:



If you would like to make the browser automatically open after run "npm run dev", go to package.json, and add "--open" after the "dev": "vite" in line 7.



Now if you run "npm run dev" the browser will open automatically

Building Item Module

Now we are going to create a simple item module. We create a file named as ItemListPage.jsx under the folder items, and change it to:

ItemListPage.jsx

```jsx
import React from 'react';
import { Table } from 'react-bootstrap';

const ItemListPage= () => {
  // Mock data
  const items = [
  ];

  return (
    <div>
      <h1>Items</h1>
      <Table striped bordered hover>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Price</th>
          <th>Descriptions</th>
          <th>Images</th>
        </tr>
      </thead>
      <tbody>
        {items.map(item => (
          <tr key={item.ItemId}>
            <td>{item.ItemId}</td>
            <td>{item.Name}</td>
            <td>{item.Price} NOK</td>
            <td>{item.Description}</td>
            <td><img src={item.ImageUrl} alt={item.Name} width="120" /></td>
          </tr>
        ))}
      </tbody>
      </Table>
    </div>
  );
};

export default ItemListPage;
```

Where the mock data look like:

```jsx
// Mock data
const items = [
  {
    ItemId: 1,
    Name: "Fried Chicken Leg",
    Price: 20,
    Description: "Crispy and succulent chicken leg that is deep-fried to perfection, often served as a popular fast food item.",
    ImageUrl: "/images/chickenleg.jpg"
  },
  {
    ItemId: 2,
    Name: "Fish and Chips",
    Price: 180,
    Description: "Classic British dish featuring battered and deep-fried fish served with thick-cut fried potatoes.",
    ImageUrl: "/images/fishandchips.jpg"
  }
];
```

We notice that there is a new package named as react-bootstrap, which contains many frequently components off-the-shelf.

Reasons to Use React-Bootstrap

- **Rapid Development**: It provides a large set of pre-built, responsive components that allow you to build a functional UI quickly.
- **Familiarity**: It's based on the popular Bootstrap framework, so you can leverage existing knowledge of a widely used system.
- **Component-Based Learning**: It's a perfect tool for learning how to use a component library, a fundamental skill in modern web development.
- **Accessibility**: The components are built with accessibility in mind, helping you learn to create inclusive web applications.

When It Is Not Suitable

While a great tool for many projects, you should not use react-bootstrap when:

- **A Unique Design is Required**: The "Bootstrap look" can be very recognisable. If a project requires a highly custom or unique visual identity, it may be better to build from scratch or use a different tool.
- **Bundle Size is a Primary Concern**: For highly optimized, performance-critical applications, the size of a full component library might be a drawback. In our project, when submitting the exam or use github, do not include node_modules (add node_modules to .gitignore) to save space.
- **Deep Customisation is Needed**: While you can customise it with CSS, it can be less flexible than a utility-first framework like Tailwind CSS for fine-grained control over every element.

For the course learning, we use the simple choice of React-Bootstrap so that we can focus on learning React. **Before installing any packages, you need to terminate the running web front-end if it is running, using Ctrl + C.** Otherwise you may encounter permission error and use admin write to clean cache. Ask AI how to handle such case.

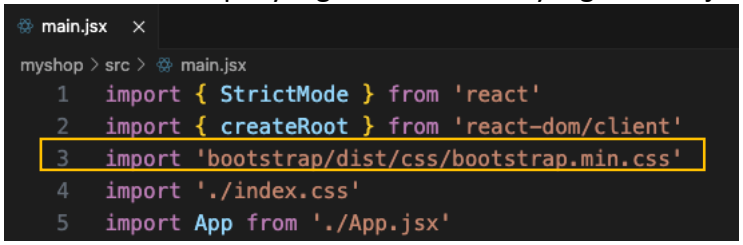
Go to the terminal under the myshop folder (which contains package.json for managing the npm packages), run:

```
baifanz@Baifan-Mac myshop % npm install react-bootstrap
```

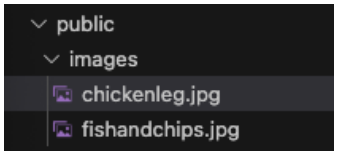
To make the style correct, we also need to install the bootstrap package:

```
baifanz@Baifan-Mac myshop % npm install bootstrap
```

Add the bootstrap styling file to use its styling in main.jsx



We also need to add the images into the folder public (for temporary mock use. Later the data will come from backend API):



Now we have our ItemListPage, but React does not know which URL points to the page. To realise this, we need the Router library that maps URLs to page elements. We do this in the next section.

Building Shared Modules, Navigation, and Routing

We install another package for building routing and navigation by running the command in myshop folder:
npm install react-router-dom

Then we change the App.jsx to this:

App.jsx

myshop > src > App.jsx > ...

```jsx
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom'
import HomePage from './home/HomePage'
import ItemListPage from './items/ItemListPage'
import './App.css'

function App() {
  return (
    <div className="App">
      <nav>
        <ul>
          <li><a href="/">Home</a></li>
          <li><a href="/items">Items</a></li>
        </ul>
      </nav>
      <Router>
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/items" element={<ItemListPage />} />
        </Routes>
      </Router>
    </div>
  )
}

export default App
```

Detailed explanations:

Imports:
BrowserRouter: This is a component from react-router-dom that provides routing functionality. It uses the HTML5 History API to keep the UI in sync with the URL. The HTML5 History API is a set of methods and properties that allow you to manipulate the browser's history and URL without causing a full page reload.
Routes: This component is used to group multiple Route components and render the first Route that matches the current URL.
Route: This component is used to define a mapping between a URL path and a component. Each Route specifies which component should be rendered when the URL matches a particular path.
HomePage and **ItemListPage**: These are custom React components that will be rendered based on the URL path.
App.css is no longer needed because we now use bootstrap for styling all pages.

Navigation element:

We have a simple navigation list that points to two links.

<Router>: This component wraps the routes in your application. It provides the routing context to the components within it.
<Routes>: This component acts as a container for the Route components. It matches the URL against its children and renders the first matching route.
<Route path="/" element={<HomePage />} />: This route renders the HomePage component when the URL path is exactly /.
<Route path="/items" element={<ItemListPage />} />: This route renders the ItemListPage component when the URL path is /items.
Now we run the project:



- Home
- Items

Welcome to My Shop



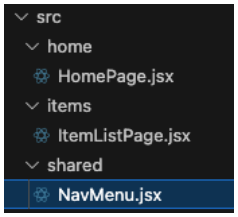
- Home
- Items

Items

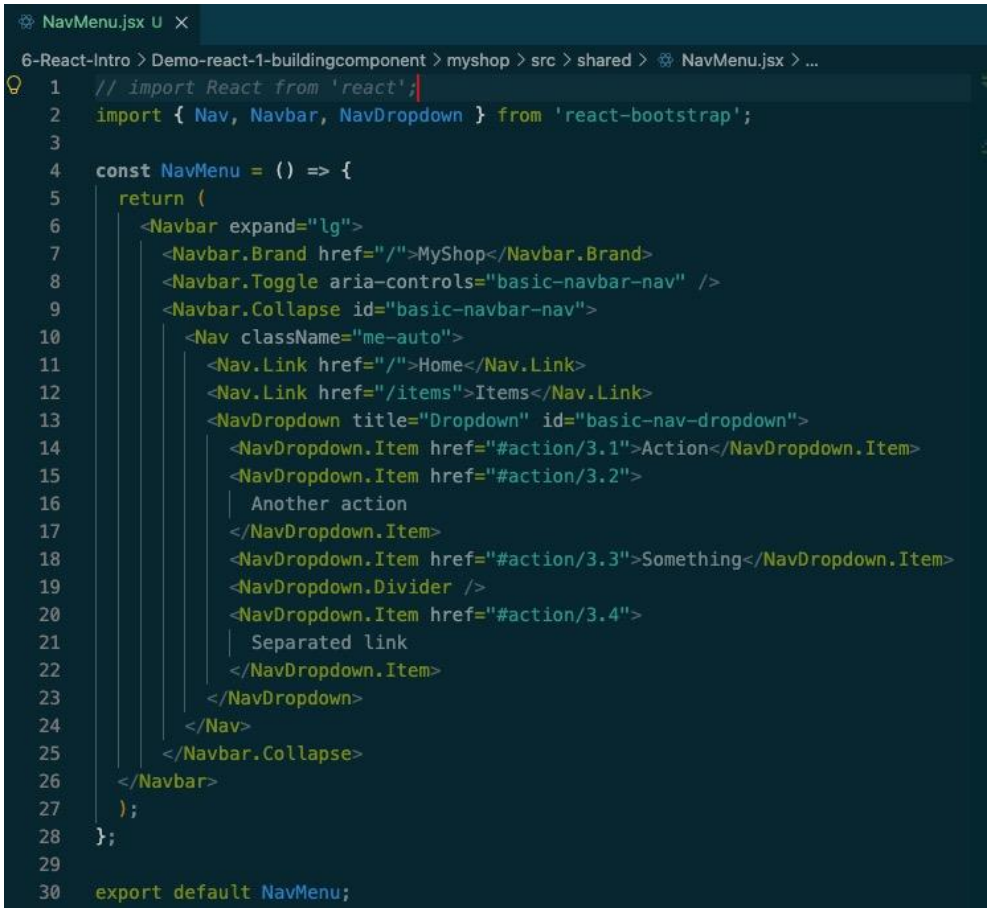
ID	Name	Price	Descriptions	Images
1	Fried Chicken Leg	20 NOK	Crispy and succulent chicken leg that is deep-fried to perfection, often served as a popular fast food item.	
2	Fish and Chips	180 NOK	Classic British dish featuring battered and deep-fried fish served with thick-cut fried potatoes.	

The navigation list does not seem good.

We create a folder named as shared, and create the file NavMenu.jsx under it:



Change the NavMenu.jsx to:



Line1 – Line2: importing the necessary libraries.
Navbar: The main container for the navigation bar.
expand="lg": The Navbar will expand (become a horizontal bar) on large screens (lg breakpoint) and collapse into a toggleable menu on smaller screens.
Navbar.Brand: Represents the brand of the website, typically shown as a logo or title. Clicking it navigates to the homepage (/). Now we are simply using the text “MyShop”.
Navbar.Toggle: Adds a button to toggle the visibility of the navigation menu on smaller screens. It provides an accessible toggle for the Navbar.Collapse.
Navbar.Collapse: Contains the collapsible content of the navigation bar. It will show or hide based on the state of the toggle button.
id="basic-navbar-nav": An identifier for the collapse behaviour, used by the Navbar.Toggle to control visibility.
Nav: Container for navigation links. The className="me-auto" applies margin-end auto spacing, pushing the navigation links to the left.
Nav.Link: Defines individual navigation links. Clicking these links navigates to different routes in the application (/ for Home and /items for Items).
NavDropdown: Creates a dropdown menu within the navigation bar.
NavDropdown.Item: Defines items within the dropdown menu. Each item is a clickable link. Now we are using fake links that point to non-existent tags. This is for demonstration purpose and will be removed later.
NavDropdown.Divider: Adds a horizontal divider between dropdown items.

Change app.js to:

App.jsx U

6-React-Intro > Demo-react-1-buildingcomponent > myshop > src > App.jsx > ...

```jsx
import { BrowserRouter as Router, Routes, Route, Navigate } from
'react-router-dom'
import Container from 'react-bootstrap/Container'
import HomePage from './home/HomePage'
import ItemListPage from './items/ItemListPage'
import NavMenu from './shared/NavMenu'
import './App.css'

function App() {

  return (
    <>
    <Container>
    <NavMenu />
        <Router>
            <Routes>
              <Route path="/" element={<HomePage />} />
              <Route path="/items" element={<ItemListPage />} />
              <Route path="*" element={<Navigate to="/" replace />} />
            </Routes>
        </Router>
    </Container>
    </>
  )
}

export default App
```

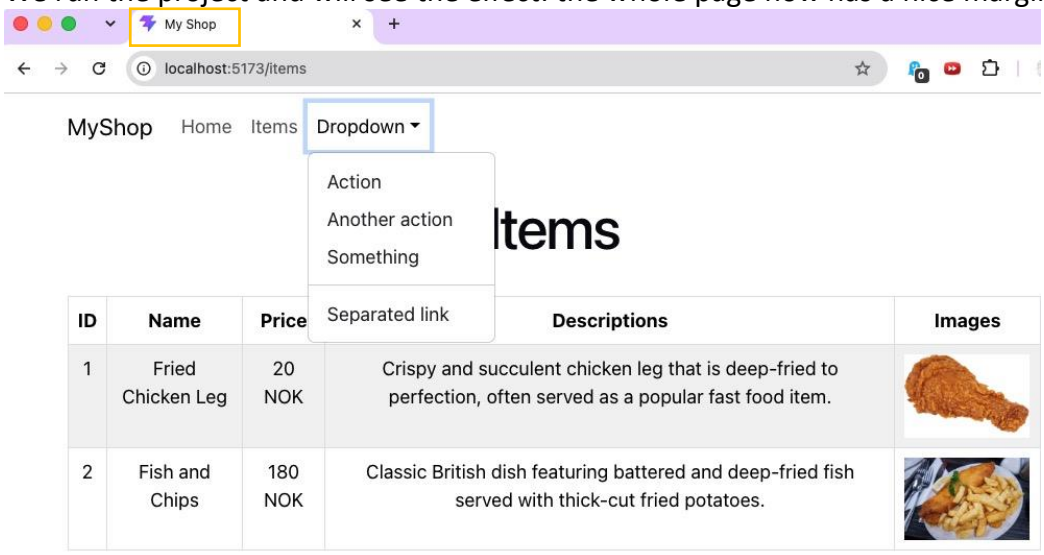
priority

<> </>: They are called a React Fragment.
Their purpose is to act as a container for multiple elements without adding an extra or other element to the HTML output. A React component must always return a single root element. If you have two elements you want to return, you can wrap them in a Fragment to satisfy this rule.
<Navigate>: it is part of the react-router-dom library and is used for programmatic navigation and redirection within a React application. It allows you to change the current URL and navigate to a different route without requiring user interaction like clicking a link.
<Container>: This is a component from react-bootstrap. It is used to centre and horizontally pad your content. The Container component provides a responsive fixed-width container that adjusts according to the screen size. It helps in maintaining consistent alignment and spacing within your application.
<Router>: This is an alias for BrowserRouter from react-router-dom, used for handling routing in your application. It keeps the UI in sync with the URL by using the HTML5 History API. This allows you to define routes and navigate between different components or pages based on the URL.
<Route path="*" element={<Navigate to="/" replace />} />: is a special configuration, which maps all unmatched URL to a Navigate element. The replace attribute tells React Router to replace the current entry (the unmatched URL) in the history stack with the new one (“/”). This means that the redirection will not create a new entry in the browser's history. As a result, pressing the back button will not return to the original undefined URL.

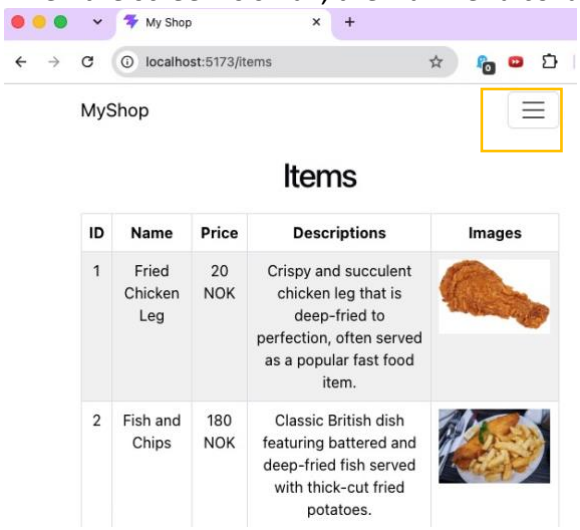
Also change the title of the webpage in index.html

```
index.html
myshop > index.html > ...
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7      <title>My Shop</title>
8    </head>
9    <body>
10     <div id="root"></div>
11     <script type="module" src="/src/main.jsx"></script>
12   </body>
13 </html>
```

We run the project and will see the effect: the whole page now has a nice margin:



When the screen is small, the NavMenu collapses the links into a button.



More options to modify the styling remain to be explored by you. ☺
Congratulations! You now have learnt the basic building blocks in React! ☺